

Keyword / Konstrukt	Purpose / Bedeutung	Mini-Beispiel
class	Definiert eine Klasse mit Attributen und Methoden	<code>class Auto { int ps; void fahren(){} }</code>
attribute / method	Attribute = Variablen der Klasse, Methoden = Funktionen	<code>int alter; void sprechen(){} </code>
new	Erstellt ein neues Objekt einer Klasse	<code>Auto a = new Auto();</code>
this	Verweist auf das aktuelle Objekt	<code>this.ps = 120;</code>
extends	Vererbung von einer anderen Klasse	<code>class Sportwagen extends Auto {}</code>
super	Zugriff auf Methoden oder Konstruktoren der Elternklasse	<code>super.fahren();</code>
import	Zugriff auf Klassen aus anderen Paketen	<code>import java.util.List;</code>
package	Organisiert Klassen in Namensräume	<code>package meineKlassen;</code>
interface	Definiert einen Vertrag mit Methoden, die implementiert werden müssen	<code>interface Fliegbar { void fliegen(); }</code>
implements	Zeigt an, dass eine Klasse ein Interface umsetzt	<code>class Vogel implements Fliegbar {}</code>
record	Kompakte, unveränderliche Datenklasse	<code>record Point(int x,int y){}</code>

Keyword / Konstrukt	Purpose / Bedeutung	Mini-Beispiel
private / protected / public	Sichtbarkeit von Attributen/Methoden	<pre>private int geheim; public void zeigen() {}</pre>
static	Mitglied gehört zur Klasse, nicht zu einem Objekt	<pre>static int counter = 0;</pre>
final	Unveränderbar / nicht überschreibbar / nicht vererbbar	<pre>final int max = 100;</pre>
abstract	Unvollständige Klasse oder Methode, muss implementiert werden	<pre>abstract class Tier { abstract void laut(); }</pre>
cast: (MyClass) x	Wandelt ein Objekt in einen bestimmten Typ um	<pre>MyClass obj = (MyClass) x;</pre>
var	Typableitung durch Compiler für lokale Variablen	<pre>var x = 5;</pre>
unnamed: _	Platzhalter für ungenutzte Werte (Pattern Matching, Lambda)	<pre>switch(obj) { case Point(_, y): ... }</pre>

Keyword / Konstrukt	Purpose / Bedeutung	Mini-Beispiel
if / else / else if	Bedingte Ausführung von Code	<code>if (a>0) { }</code>
for	Schleife mit Anfang, Bedingung und Schrittweite	<code>for(int i=0;i<10;i++){ }</code>
for-i	Kurzform für Index-basierte Schleife	<code>for(int i=0;i<n;i++){ }</code>
while	Schleife, die läuft, solange Bedingung wahr ist	<code>while (x>0) { }</code>
continue	Überspringt aktuellen Schleifendurchlauf	<code>if (x<0) continue;</code>
break	Beendet Schleife oder switch sofort	<code>if (x==0) break;</code>
switch	Wählt zwischen mehreren Alternativen	<code>switch(tag){ case 1: ... }</code>
default	Standardfall in switch	<code>default: System.out.println("X");</code>
return	Gibt einen Wert aus einer Methode zurück und beendet sie	<code>return x + y;</code>
void	Methode gibt keinen Wert zurück	<code>void hallo() { }</code>

Keyword / Konstrukt

Purpose / Bedeutung

Mini-Beispiel

try-catch-finally / throw / throws

Ausnahmebehandlung

```
try{...}catch(Exception  
e){...}
```

instanceof

Prüft, ob Objekt Instanz eines
Typs ist

```
if(obj instanceof  
Auto){}
```

[]

Deklariert oder greift auf ein Array
zu

```
int[] arr = new int[5];
```